Superior String Splitting

Document #:	P2210R1
Date:	2021-01-04
Project:	Programming Language C++
Audience:	LEWG

Reply-to:	Barry Revzin <barry.revzin@gmail.com>

Contents

§ 1 Revision History

Since [P2210R0], corrected the explanation of `const`-iteration and corrected that it is sort of possible to build a better `split` on top of the existing one. Changed the proposal to keep the preexisting `split` with its semantics under a different name instead.

§ 2 Introduction

Let's say you have a string like `"1.2.3.4"` and you want it turn it into a range of integers. You might expect to be able to write:

```
std::string s = "1.2.3.4";

auto ints =
    s | views::split('.')
      | views::transform([](auto v){
            int i = 0;
            std::from_chars(v.data(), v.size(), &i);
            return i;
        })
```

But that doesn't work. Nor does this:

```
std::string s = "1.2.3.4";

auto ints =
    s | views::split('.')
      | views::transform([](auto v){
          return std::stoi(std::string(v.begin(), v.end()));
        });
}
```

although for a different reason.

The problem ultimately is that splitting a `string` using C++20's `views::split` gives you a range that is *only* a forward range. Even though the source range is contiguous! And that forward range isn't a common range either. As a result, can't use `from_chars` (it needs a pointer) and can't construct a `std::string` (its range constructor takes an iterator pair, not iterator/sentinel).

2.1 Can we get there from here

The reason it's like this is that `views::split` is maximally lazy. It kind of needs to be in order to support splitting an input range. But the intent of laziness is that you can build more eager algorithms on top of them. We can sort of do that with this one:

```
std::string input = "1.2.3.4";
auto parts = input | views::split('.')
                   | views::transform([](auto r){
                       auto b = r.begin();
                       auto e = ranges::next(b, r.end());
                       return ranges::subrange(b.base(), e.base());
                     });
```

This implementation requires `b.base()` (`split_view`'s iterators do not provide such a thing at the moment), but once we add that, this solution provides the right overall shape that we need. The individual elements of `parts` here are contiguous views, which will have a `data()` member function that can be passed to `std::from_chars`.

Given the range adapter closure functionality, we could just stash this somewhere:

```
template <typename Pattern>
auto split2(Pattern pattern) {
    return views::split(std::move(pattern))
```

```
        | views::transform([](auto r){
            auto b = r.begin();
            auto e = ranges::next(b, r.end());
            return ranges::subrange(b.base(), e.base());
          });
}

std::string input = "1.2.3.4";
auto parts = input | split2('.');
```

Is this good enough?

It's better, but there's some downsides with this. First, the above fares pretty badly with input iterators - since we've already consumed the whole range before even providing it to the user. If they touch the range in any way, it's undefined behavior. The narrowest possible contract?

We can fix that:

```
template <typename Pattern>
auto split2(Pattern pattern) {
    return views::split('.')
        | views::transform([](auto r){
              if constexpr (ranges::forward_range<decltype(r)>) {
                  auto b = r.begin();
                  auto e = ranges::next(b, r.end());
                  return ranges::subrange(b.base(), e.base());
              } else {
                  return r;
              }
          });
}
```

Now we properly handle input ranges as well, although we can't really avoid the extra completely-pointless `transform` without writing a proper range adapter closure ourselves.

Second, this has the problem that every iterator dereference from the resulting view has to do a linear search. That's a cost that's incurred entirely due to this implementation strategy. This cost could be alleviated by using [P2214R0]'s suggested `views::cache_latest`, though this itself has the problem that it demotes the resulting range to input-only… whereas a `split_view` could be a forward range.

Third, this is complicated! There are two important points here:

1. `split` is overwhelmingly likely to be used on, specifically, a contiguous range of characters.
2. `split`ting a `string` is a common operation to want to do.

While I've `transform`ed and `filter`ed all sorts of other kinds of ranges, and have appreciated the work that went into making them as flexible as they are, I've really never wanted to split anything other than a `string` (or a `string_view` or other equivalent types). But for the most common use case of a fairly common operation, the `views::split` that we have ends up falling short because of what it gives us: a forward range. Pretty much every interesting string algorithm requires more than that:

— The aforementioned `std::from_chars` requires a contiguous range (really, specifically, a `char const*` and a length)

— `std::regex` and `boost::regex` both require a bidirectional range.

— Many text and unicode algorithms require a bidirectional range.

It would be great if `views::split` could work in such a way that the result was maximally usable - which in this case means that splitting a contiguous range should provide contiguous sub-ranges.

3 Design

I wrote a blog on this topic [revzin.split], which implements a version of `split_view` that operates on contiguous ranges (and naughtily partially specializes `std::ranges::split_view`) such that the following actually works:

```
auto ip = "127.0.0.1"s;
auto parts = ip | std::views::split('.')
                | std::views::transform([](std::span<char const> s){
                    int i;
                    std::from_chars(s.data(), s.data() + s.size(), i);
                    return i;
                  });

struct zstring_sentinel {
    bool operator==(char const* p) const {
        return *p == '\0';
    }
};

struct zstring : view_interface<zstring> {
    char const* p = nullptr;
    zstring() = default;
    zstring(char const* p) : p(p) { }
    auto begin() const { return p; }
    auto end() const { return zstring_sentinel{}; }
};

char const* words = "A quick brown fox";
for (std::span ss : zstring{words} | std::views::split(' ')) {
    auto sv = std::string_view(ss.data(), ss.size());
    std::cout << sv << '\n';
}
```

There are a few big questions to be resolved here:

3.1 What should the reference type of this be?

Given a range `V`, the reference type of `split_view<V>` should be `subrange<iterator_t<V>>`. This is a generic solution, that works well for all range categories (as long as they're forward-or-better).

For splitting a `string`, this means we get a range of `subrange<string::iterator>` where we might wish we got a `span<char const>` or a `string_view`, but `span<char const>` is already constructible from `subrange<string::iterator>` and `string_view` would be with the adoption of [P1989R0].

We could additionally favor the `char` case (since, again, splitting `strings` is the overwhelmingly common case) by doing something like:

```
struct reference : subrange<iterator_t<V>> {
    using subrange::subrange;

    operator string_view() const
        requires same_as<range_value_t<V>, char>
              && contiguous_range<V>
    {
        return {this->data(), this->size()};
    }
};
```

But this just seems weirdly specific and not a good idea.

§ 3.2 How would `const` iteration work?

We say that a type, `R`, is `const`-iterable if `R const` is also a range. This comes into play if you want to write something like:

```
// parts is 'const'
auto const parts = rng | views::split(pat);
```

Or pass this adapted range into a function template that looks like this:

```
template <typename R> void some_algo(R const&);
```

Several range adapters in the standard library today are *not* `const`-iterable (such as `filter_view` and its similar cousin `drop_while_view`). See also [issue385].

The big issue for moving forward with `std::ranges::split_view` is how to square some constraints:

1. `begin()` must be amortized constant time to model `range`
2. Member functions that are marked `const` in the standard library cannot introduce data results (see 16.4.6.10 [res.on.data.races]).
3. The existing `split_view` is `const`-iterable.

So how do we get the first piece? What I did as part of my implementation was to not allow `const`-iteration. `split_view` just has an `optional<iterator>` member which is populated the first time you call `begin` and then is just returned thereafter. This is similar to other views that need to do arbitrary work to yield the first element (canonically, `filter_view`).

You can't do this in a `const` member function. We cannot simply specify the `optional<iterator>` member as `mutable`, since two threads couldn't safely update it concurrently. We *could* introduce a

synchronization mechanism into `split_view` which would safely allow such concurrent modification, but that's a very expensive solution which would also pessimize the typical non-const-iteration case. So we *shouldn't*.

So that's okay, we just don't support `const`-iteration. What's the big deal?

Well, as I noted, the *existing* `split_view` *is* `const`-iterable - because it's lazy! It doesn't yield `subranges`, it yields a lazy range. So it doesn't actually need to do work in `begin()` - this is a non-issue.

On the other hand, any kind of implementation that eagerly produces `subranges` for splitting a `string const` necessarily has to do work to get that first `subrange` and that ends up being a clash with the existing design.

The question is - how much code currently exists that iterates over, specifically, a `const split_view` that is splitting a contiguous range? It's likely to be exceedingly small, both because of the likelihood if such iteration to begin with (you'd have to *specifically* declare an object of type `split_view` as being `const`) and the existing usability issues described in this paper.

Plus, at this point, only libstdc++ ships `split_view`. And even then, it implements the spec to the letter, which means there are at least two issues with it: broken support of long patterns [LWG3505] and trailing patterns [LWG3478]. So even if somebody is depending on existing behavior, they probably shouldn't.

Personally, I think the trade-off is hugely in favor of making this change - it makes `split` substantially more useful.

§ 3.3 What about input ranges?

This strategy of producing a range of `subrange<iterator_t<V>>` works fine for forward ranges, and is a significant improvement over status quo for bidirectional, random access, and contiguous ranges.

But it fails miserably for input ranges, which the current `views::split` supports. While `const`-iteration doesn't strike me as important functionality, being able to `split` an input range definitely does.

A `subrange`-yielding split could still fall back to the existing `views::split` behavior for splitting input ranges, but then we end up with fairly different semantics for splitting input ranges and splitting forward-or-better ranges. But we also don't want to *not* support splitting input ranges.

This leads us to the big question…

§ 3.4 Replace or Add

Do we *replace* the existing `views::split` with the one outlined in this paper, or do we add a new one?

The only way to really replace is if we have the one-view-two-semantics implementation described in the previous section: for forward-or-better, the view produces `subranges`, while for input, it produces a lazy-searching range. This just doesn't seem great from a design perspective. And if we *don't* do that, then the only way we can replace is by dropping input range support entirely. Which doesn't seem great from a

user perspective since we lose functionality. Granted, we gain the ability to actually split strings well, which is drastically more important than splitting input ranges, but still.

The alternative is to add a new kind of `views::split` that only supports splitting forward-or-better ranges. We could adopt such a new view under a different name (`views::split2` is the obvious choice, but `std2::split` is even better), but I think it would be better to give the good name to the more broadly useful facility.

That is, provide a new `views::split` that produces `subranges` and rename the existing facility to `views::lazy_split`. This isn't a great name, since `views::split` is *also* a lazy split, but it's the best I've got at the moment.

Note that, as described earlier, `views::split` should not be specified (or implemented) in terms of `views::lazy_split`, since a proper implementation of it could be much more efficient.

§ 4 Proposal

This paper proposes the following:

1. Rename the existing `views::split` / `ranges::split_view` to `views::lazy_split` / `ranges::lazy_split_view`. Add `base()` member functions to the *`inner-iterator`* type to get back to the adapted range's iterators.

2. Introduce a new range adapter under the name `views::split` / `ranges::split_view` with the following design:

    a. It can only support splitting forward-or-better ranges.

    b. Splitting a `V` will yield `subrange<iterator_t<V>>`s, ensuring that the adapted range's category is preserved. Splitting a bidirectional range gives out bidirectional subranges. Spltiting a contiguous range gives out contiguous subranges.

    c. `views::split` will not be `const`-iterable.

This could certainly break some C++20 code. But I would argue that `views::split` is so unergonomic for its most common intended use-case that the benefit of making it actually usable for that case far outweighs the cost of potentially breaking some code (which itself is likely to be small given both the usability issues and lack of implementations thus far).

§ 4.1 Implementation

The implementation can be found in action here [revzin.split.impl], but reproduced here for clarity.

This implementation does the weird `operator string_view() const` hack described earlier as a workaround for the lack of [P1989R0], and currently just hijacks the real `views::split` as a shortcut, but other than that it should be a valid implementation.

This implementation also correctly handles [LWG3505] and [LWG3478].

```
using namespace std::ranges;

template <forward_range V, forward_range Pattern>
    requires view<V> && view<Pattern> &&
    std::indirectly_comparable<iterator_t<V>,
                               iterator_t<Pattern>,
                               equal_to>
class split2_view
    : public view_interface<split2_view<V, Pattern>>
{
public:
    split2_view() = default;
    split2_view(V base, Pattern pattern)
        : base_(base)
        , pattern_(pattern)
    { }

    template <forward_range R>
        requires std::constructible_from<V, views::all_t<R>>
            && std::constructible_from<Pattern, single_view<range_value_t<R>>>
    split2_view(R&& r, range_value_t<R> elem)
        : base_(std::views::all(std::forward<R>(r)))
        , pattern_(std::move(elem))
    { }

    struct sentinel;
    struct as_sentinel_t { };

    class iterator {
    private:
        friend sentinel;

        split2_view* parent = nullptr;
        iterator_t<V> cur = iterator_t<V>();
        iterator_t<V> next = iterator_t<V>();
        bool trailing_empty = false;

    public:
        iterator() = default;
        iterator(split2_view* p)
            : parent(p)
            , cur(std::ranges::begin(p->base_))
            , next(lookup_next())
        { }

        iterator(as_sentinel_t, split2_view* p)
            : parent(p)
            , cur(std::ranges::end(p->base_))
            , next()
        { }

        using iterator_category = std::forward_iterator_tag;
        struct reference : subrange<iterator_t<V>> {
            using reference::subrange::subrange;
```

```
        operator std::string_view() const
            requires std::same_as<range_value_t<V>, char>
                  && contiguous_range<V>
        {
            return {this->data(), this->size()};
        }
    };
    using value_type = reference;
    using difference_type = std::ptrdiff_t;

    bool operator==(iterator const& rhs) const {
        return cur == rhs.cur && trailing_empty == rhs.trailing_empty;
    }

    auto lookup_next() const -> iterator_t<V> {
        auto n = std::ranges::search(
            subrange(cur, std::ranges::end(parent->base_)),
            parent->pattern_
            ).begin();

        if (n != std::ranges::end(parent->base_) and std::ranges::empty(parent-
    >pattern_)) {
            ++n;
        }

        return n;
    }

    auto operator++() -> iterator& {
        cur = next;
        if (cur != std::ranges::end(parent->base_)) {
            std::ranges::advance(cur, distance(parent->pattern_));
            if (cur == std::ranges::end(parent->base_)) {
                trailing_empty = true;
            }
            next = lookup_next();
        } else {
            trailing_empty = false;
        }
        return *this;
    }
    auto operator++(int) -> iterator {
        auto tmp = *this;
        ++*this;
        return tmp;
    }

    auto operator*() const -> reference {
        return {cur, next};
    }
};

struct sentinel {
    bool operator==(iterator const& rhs) const {
        return rhs.cur == sentinel and not rhs.trailing_empty;
    }
```

```
        sentinel_t<V> sentinel;
    };


    auto begin() -> iterator {
        if (not cached_begin_) {
            cached_begin_.emplace(this);
        }
        return *cached_begin_;
    }
    auto end() -> sentinel {
        return {std::ranges::end(base_)};
    }

    auto end() -> iterator requires common_range<V> {
        return {as_sentinel_t(), this};
    }

private:
    V base_ = V();
    Pattern pattern_ = Pattern();
    std::optional<iterator> cached_begin_;
};

// It's okay if you're just writing a paper?
namespace std::ranges {
    template<forward_range V, forward_range Pattern>
    requires view<V> && view<Pattern>
      && indirectly_comparable<iterator_t<V>, iterator_t<Pattern>, equal_to>
    class split_view<V, Pattern> : public split2_view<V, Pattern>
    {
        using split2_view<V, Pattern>::split2_view;
    };
}
```

§ 5 Wording

Change 24.2 [ranges.syn] to add the new view:

```
  // [range.split], split view
  template<class R>
    concept tiny-range = see below;   // exposition only

  template<input_range V, forward_range Pattern>
    requires view<V> && view<Pattern> &&
             indirectly_comparable<iterator_t<V>, iterator_t<Pattern>,
         ranges::equal_to> &&
             (forward_range<V> || tiny-range<Pattern>)
- class split_view;
+ class lazy_split_view;

+ template<forward_range V, forward_range Pattern>
+   requires view<V> && view<Pattern> &&
```

```
+            indirectly_comparable<iterator_t<V>, iterator_t<Pattern>,
        ranges::equal_to>
+ class split_view;

- namespace views { inline constexpr unspecified split = unspecified; }
+ namespace views {
+   inline constexpr unspecified lazy_split = unspecified;
+   inline constexpr unspecified split = unspecified;
+ }
```

5.1 Wording for `split` -> `lazy_split`

Rename all references to `split` and `split_view` in 24.7.12 [range.split] to `lazy_split` and `lazy_split_view`, respectively, and rename all the clauses from [range.split.meow] to [range.lazy.split.meow].

Add `base()` overloads to '`lazy_split_view::inner-iterator`' in 24.7.12.5 [range.split.inner]:

```
namespace std::ranges {
  template<input_range V, forward_range Pattern>
    requires view<V> && view<Pattern> &&
             indirectly_comparable<iterator_t<V>, iterator_t<Pattern>,
         ranges::equal_to> &&
             (forward_range<V> || tiny-range<Pattern>)
  template<bool Const>
- struct split_view<V, Pattern>::inner-iterator {
+ struct lazy_split_view<V, Pattern>::inner-iterator {
  private:
    using Base = maybe-const<Const, V>;                  // exposition only
    outer-iterator<Const> i_ = outer-iterator<Const>(); // exposition only
    bool incremented_ = false;                           // exposition only
  public:
    using iterator_concept  = typename outer-
        iterator<Const>::iterator_concept;
    using iterator_category = see below;
    using value_type        = range_value_t<Base>;
    using difference_type   = range_difference_t<Base>;

    inner-iterator() = default;
    constexpr explicit inner-iterator(outer-iterator<Const> i);

+   constexpr iterator_t<Base> base() const&
+       requires copyable<iterator_t<Base>>;
+   constexpr iterator_t<Base> base() &&;

    constexpr decltype(auto) operator*() const { return *i_.current; }

    constexpr inner-iterator& operator++();
    constexpr decltype(auto) operator++(int) {
      if constexpr (forward_range<V>) {
        auto tmp = *this;
        ++*this;
        return tmp;
      } else
```

```
        ++*this;
    }

    friend constexpr bool operator==(const inner-iterator& x, const inner-
        iterator& y)
      requires forward_range<Base>;

    friend constexpr bool operator==(const inner-iterator& x,
        default_sentinel_t);

    friend constexpr decltype(auto) iter_move(const inner-iterator& i)
    noexcept(noexcept(ranges::iter_move(i.i_.current))) {
      return ranges::iter_move(i.i_.current);
    }

    friend constexpr void iter_swap(const inner-iterator& x, const inner-
        iterator& y)
      noexcept(noexcept(ranges::iter_swap(x.i_.current, y.i_.current)))
      requires indirectly_swappable<iterator_t<Base>>;
  };
}
```

And add text describing those two new functions, just before `operator++`:

```
constexpr iterator_t<Base> base() const&
    requires copyable<iterator_t<Base>>;
```

a *Effects*: Equivalent to: `return i_.current;`

```
constexpr iterator_t<Base> base() &&;
```

b *Effects*: Equivalent to: `return std::move(i_.current);`

```
constexpr inner-iterator& operator++();
```

3 *Effects*: […]

§ 5.2 Wording for new `split`

[Editor's note: All of the wording here is new, so I'm not going to highlight it in green. To help differentiate that these are all new clauses, I'm going to refer to them as [range.split2] but the intent here is that they would be added under the name [range.split] throughout]

§ 5.2.1 Overview [range.split2.overview]

1 `split_view` takes a `view` and a delimiter, and splits the `view` into `subranges` on the delimiter. The delimiter can be a single element or a `view` of elements. [Editor's note: NB: code font on `subrange` since they are, specifically, `subranges`]

2 The name `views::split` denotes a range adaptor object ([range.adaptor.object]). Given subexpressions `E` and `F`, the expression `views::split(E, F)` is expression-equivalent to `split_view(E, F)`.

3 [*Example 1*:

```
string str{"the quick brown fox"};
for (span<char const> word : split(str, ' ')) {
  for (char ch : word)
    cout << ch;
  cout << '*';
}
// The above prints: the*quick*brown*fox*
```

[Editor's note: Same example as we have for split today, except explicitly iterating via span]

— *end example*]

§ 5.2.2 Class template `split_view` [range.split2.view]

```
template<forward_range V, forward_range Pattern>
  requires view<V> && view<Pattern> &&
           indirectly_comparable<iterator_t<V>, iterator_t<Pattern>,
       ranges::equal_to>
class split_view : public view_interface<split_view<V, Pattern>> {
private:
  V base_ = V();                                  // exposition only
  Pattern pattern_ = Pattern();                   // exposition only
  // [range.split2.iterator], class split_view::iterator
  struct iterator;                                // exposition only
  // [range.split2.sentinel], class split_view::sentinel
  struct sentinel;                                // exposition only
public:
  split_view() = default;
  constexpr split_view(V base, Pattern pattern);

  template<forward_range R>
    requires constructible_from<V, views::all_t<R>> &&
             constructible_from<Pattern, single_view<range_value_t<R>>>
  constexpr split_view(R&& r, range_value_t<R> e);

  constexpr V base() const { return base_; }

  constexpr iterator begin();

  constexpr auto end() {
      if constexpr (common_range<V>) {
          return iterator{*this, ranges::end(base_), {}};
      } else {
          return sentinel{*this};
      }
  }

  constexpr iterator_t<V> find-next(iterator_t<V>); // exposition only
};

template<class R, class P>
  split_view(R&&, P&&) -> split_view<views::all_t<R>, views::all_t<P>>;
```

```
  template<forward_range R>
    split_view(R&&, range_value_t<R>)
      -> split_view<views::all_t<R>, single_view<range_value_t<R>>>;
}
```

```
constexpr split_view(V base, Pattern pattern);
```

1 *Effects*: Initializes *base_* with `std::move(base)`, and *pattern_* with `std::move(pattern)`.

```
template<forward_range R>
  requires constructible_from<V, views::all_t<R>> &&
           constructible_from<Pattern, single_view<range_value_t<R>>>
constexpr split_view(R&& r, range_value_t<R> e);
```

2 *Effects*: Initializes *base_* with `views::all(std::forward<R>(r))`, and *pattern_* with `single_view{std::move(e)}`.

```
constexpr iterator begin();
```

3 *Returns*: `{*this, ranges::begin(base_), find-next(ranges::begin(base_))}`.

4 *Remarks*: In order to provide the amortized constant time complexity required by the `range` concept, this function caches the result within the `split_view` for use on subsequent calls.

```
constexpr subrange<iterator_t<V>> find-next(iterator_t<V> it); // exposition
        only
```

5 *Effects*: Equivalent to:

```
auto [b,e] = ranges::search(subrange(it, ranges::end(base_)), pattern_);
if (b != ranges::end(base_) && ranges::empty(pattern_)) {
    ++b;
}
return {b,e};
```

§ 5.2.3 Class template `split_view::iterator` [range.split2.iterator]

```
  template<forward_range V, forward_range Pattern>
    requires view<V> && view<Pattern> &&
             indirectly_comparable<iterator_t<V>, iterator_t<Pattern>,
        ranges::equal_to>
  class split_view<V, Pattern>::iterator {
  private:
    split_view* parent_ = nullptr;                                  //
        exposition only
    iterator_t<V> cur_ = iterator_t<V>();                           //
        exposition only
    subrange<iterator_t<V>> next_ = subrange<iterator_t<V>>();  //
        exposition only
    bool trailing_empty_ = false;                                   //
        exposition only

  public:
    using iterator_concept = forward_iterator_tag;
```

```
    using iterator_category = input_iterator_tag;
    using value_type = subrange<iterator_t<V>>;
    using difference_type = range_difference_t<V>;

    iterator() = default;
    constexpr iterator(split_view& parent, iterator_t<V> current,
        subrange<iterator_t<V>> next);

    constexpr iterator_t<V> base() const;
    constexpr value_type operator*() const;

    constexpr iterator& operator++();
    constexpr iterator operator++(int);

    friend constexpr bool operator==(const iterator& x, const iterator& y)
  };
```

```
constexpr iterator(split_view& parent, iterator_t<V> current,
        subrange<iterator_t<V>> next);
```

1 *Effects*: Initializes *parent_* with `addressof(parent)`, *cur_* with `std::move(current)`, and *next_* with `std::move(next)`.

```
constexpr iterator_t<V> base() const;
```

2 *Effects*: Equivalent to `return cur_;`

```
constexpr value_type operator*() const;
```

3 *Effects*: Equivalent to `return {cur_, next_.begin()};`

```
constexpr iterator& operator++();
```

4 *Effects*: Equivalent to:

```
cur_ = next_.begin();
if (cur_ != ranges::end(parent_->base_)) {
    cur_ = next_.end();
    if (cur_ == ranges::end(parent_->base_)) {
        trailing_empty_ = true;
        next_ = {cur_, cur_};
    } else {
        next_ = parent_->find-next(cur_);
    }
} else {
    trailing_empty_ = false;
}
return *this;
```

```
constexpr iterator operator++(int);
```

5 *Effects*: Equivalent to:

```
auto tmp = *this;
++*this;
```

```
return tmp;
```

```
friend constexpr bool operator==(const iterator& x, const iterator& y)
```

6 *Effects*: Equivalent to:

```
return x.cur_ == y.cur_ && x.trailing_empty_ == y.trailing_empty_;
```

§ 5.2.4 Class template `split_view::sentinel` [range.split2.sentinel]

```
template<forward_range V, forward_range Pattern>
  requires view<V> && view<Pattern> &&
           indirectly_comparable<iterator_t<V>, iterator_t<Pattern>,
       ranges::equal_to>
struct split_view<V, Pattern>::sentinel {
private:
  sentinel_t<V> end_ = sentinel_t<V>(); // exposition only

public:
  sentinel() = default;
  constexpr explicit sentinel(split_view& parent);

  friend constexpr bool operator==(const iterator& x, const sentinel& y);
};
```

```
constexpr explicit sentinel(split_view& parent);
```

1 *Effects*: Initializes *end_* with `ranges::end(parent.base_)`.

```
friend constexpr bool operator==(const iterator& x, const sentinel& y);
```

2 *Effects*: Equivalent to: `return x.cur_ == y.end_ && !x.trailing_empty_;`

§ 6 Acknowledgments

Thanks to Tim Song for help with implementation and wording.

§ 7 References

[issue385] Casey Carter. 2016. `const`-ness of view operations.
https://github.com/ericniebler/range-v3/issues/385

[LWG3478] Barry Revzin. views::split drops trailing empty range.
https://wg21.link/lwg3478

[LWG3505] Tim Song. split_view::outer-iterator::operator++ misspecified.
https://wg21.link/lwg3505

[P1989R0] Corentin Jabot. 2019-11-25. Range constructor for std::string_view 2: Constrain Harder.
https://wg21.link/p1989r0

[P2210R0] Barry Revzin. 2020-08-14. Superior String Splitting.
https://wg21.link/p2210r0

[P2214R0] Barry Revzin, Conor Hoekstra, Tim Song. 2020-10-15. A Plan for C++23 Ranges.
https://wg21.link/p2214r0

[revzin.split] Barry Revzin. 2020. Implementing a better views::split.
https://brevzin.github.io/c++/2020/07/06/split-view/

[revzin.split.impl] Barry Revzin. 2020. Implementation of `split2_view`.
https://godbolt.org/z/hanc46